Beginner Guide For Presenting The Paper

Multi-Solution Search and the Limits of Neural Guidance in Rubik's Cube Solving

Akash Chakraborty Atal Chaudhuri

Sister Nivedita University, Kolkata

April 28, 2026

How to use this guide. Read it straight through once without trying to memorise anything. Every term in **bold** is explained the first time it appears. On second reading, focus on Sections 33–35: those contain the memorisable lines and the viva answers. You do not need the paper in front of you to follow this guide; every number quoted here is cross-referenced to the paper's frozen result artifacts.

Contents

1 What is this paper about?

This paper is about making a Rubik's Cube solving program better.

The authors start with an existing, very well-known solver called **Kociemba's Two-Phase Algorithm**, and then add three extensions:

1. **Multi-solution search.** Try several possible halfway states instead of only the first one, and keep the shortest full solution.
2. **Anytime mode.** Stop early if the caller only has a limited time budget (useful for interactive apps).
3. **Neural move ordering.** Use a small AI model to guess which move should be tried first during search.

The **biggest success is the first one**. The anytime mode is a simple engineering knob. The AI idea is interesting and carefully studied, but the paper's own finding is that it does **not** help the final solver much. That negative result is itself a main contribution of the paper.

2 What is a solver?

A **solver** is a program that takes a scrambled Rubik's Cube and returns a sequence of moves that solves it.

> *Input: a scrambled cube.* *Output: e.g.* `R U R' U'`.

That output means: turn the Right face clockwise, then the Up face clockwise, then the Right face counter-clockwise, then the Up face counter-clockwise. A cube has 18 possible moves at every step (six faces × three turn types), so the space of possible sequences is astronomically large.

3 How big is the space?

A 3×3 Rubik's Cube has about 4.3×10^{19} reachable states. That is 43 billion billion. You cannot brute-force search that; you need a smart algorithm.

4 What is Kociemba's Two-Phase Algorithm?

This is the classical algorithm the paper builds on. It solves the cube in **two steps** rather than in one shot.

Phase 1. The solver does not fully solve the cube. It only brings the cube into a special *halfway group* called G_1. You can think of G_1 as the set of cube states where all edges are correctly oriented, all corners are correctly oriented, and the middle-layer edges are in the middle layer. The cube is still scrambled, but in a more structured way.

Phase 2. From that halfway state, the solver finishes the cube, but now using only 10 of the 18 moves (the other 8 would break the structure Phase 1 just established). The reduced set of moves means the search is much easier.

Why do this in two phases? Because each phase has a much smaller effective search space than solving in one shot. Phase 2 in particular becomes very fast.

5 What is IDA*?

Both phases use a search algorithm called **IDA*** (Iterative Deepening A*). Intuitively:

- The solver tries to find a solution at depth 1. If none, depth 2. If none, depth 3, and so on.
- At each depth limit, it does a depth-first search, using a **heuristic** (see Section 7) to prune branches that cannot possibly succeed.

You do not need the formal definition for the presentation. Just remember:

> *IDA* is the tree-search procedure inside Kociemba's two phases. It expands nodes one depth at a time, guided by a lower-bound heuristic.*

6 What is a search tree, and what is a "node"?

Imagine a tree where:

- the root is the scrambled cube,
- each child is what the cube looks like after one move,
- grandchildren are what it looks like after two moves, and so on.

Each point in that tree is a **node**. When the paper says "expanded 6.79M nodes", it means the solver inspected about 6.79 million cube positions on its way to finding a solution.

More nodes = more work = more time. Fewer nodes is better.

7 What is a heuristic?

A **heuristic** is a fast guess about how far a cube state is from being solved. In Kociemba, this guess is a *lower bound*: it never overestimates. IDA* uses it to throw away branches that cannot finish in the current depth limit.

8 What is a pruning table?

A **pruning table** is a giant precomputed lookup table that stores the minimum number of moves needed from each compact cube coordinate down to the solved state (or to the halfway G_1 state in Phase 1). Kociemba uses a few of these tables, totalling about 20 MB, built once and saved to disk.

At every node during search, the solver just reads a few numbers from these tables. No actual calculation. That is what makes Kociemba so fast: the hard work was done once, offline.

9 What does "pruning" mean?

Pruning means cutting away useless branches of the search tree early, before exploring them. If the pruning table says "this branch cannot reach the goal within the current depth limit", the solver skips it.

10 What is move ordering?

At any given node, the solver has up to 18 legal moves it can try next. **Move ordering** is the decision of *which order* to try those moves in. If the first move you try is the right move, you find the solution quickly. If you always try the wrong ones first, you waste time. Kociemba's default ordering is a fixed lexicographic order $(U, U', U_2, R, R', \ldots)$.

The neural part of this paper is an attempt to replace that fixed order with a learned order.

11 What is the paper's main algorithmic idea?

Normal Kociemba:

- Finds the first Phase 1 endpoint.
- Runs Phase 2 from there.

- Returns the combined sequence.

This paper:

- Finds up to K different Phase 1 endpoints.
- Runs Phase 2 from each.
- Returns the shortest combined solution.

That is it. Nothing clever, nothing fancy. And it works surprisingly well in practice.

12 Why does that help?

Two different Phase 1 endpoints are both "valid halfway points", but one of them might sit much closer to the solved cube than the other. The default Kociemba always accepts the first endpoint it finds, even if a slightly later one would have led to a shorter Phase 2. Trying K endpoints and keeping the best is like taking the *minimum of K draws* from the Phase 2-length distribution.

If you remember one intuition, remember this:

> *"Don't stop at the first halfway state you find; try a few and pick the one that leads to the shortest finish."*

13 What does "K=5" mean?

K is just the number of Phase 1 endpoints the solver will try.

- $K = 1$ is the original Kociemba behaviour (one endpoint).
- $K = 3$ is a middle setting: tries three endpoints.
- $K = 5$ is the paper's default recommended setting.
- $K = 10$ is more exhaustive but runs into diminishing returns.

Bigger K usually means better solution quality but more computation time. The paper's data shows that the gain saturates around $K = 3$ to $K = 5$.

14 What is the headline result?

On the canonical 200-cube, seed-42 benchmark, all reported numbers come from the frozen file `results/benchmark_200_retrained.json`:

Setting	**Mean moves**	**≤20 (%)**	**Total nodes**	**Mean time**
Baseline ($K = 1$)	20.77	28.0%	788 K	582 ms
Multi ($K = 5$)	20.23	57.5%	6.79 M	5103 ms
$K = 5$ + LUT	20.27	54.8%	6.02 M	5009 ms

So multi-solution search:

- Reduces mean solution length by 0.53 moves (20.77 → 20.23).
- Roughly **doubles** the fraction of ≤20-move solutions (28.0% → 57.5%).
- Costs about 8.6× more expanded nodes (788 K → 6.79 M).

That is the paper's strongest, cleanest result.

15 What does "≤20 moves" mean, and why is it important?

≤20 just means "20 moves or fewer". This cut-off matters because of **God's Number**. See the next section.

16 What is God's Number?

God's Number is the fewest moves needed to solve *any* Rubik's Cube position if you use the best possible solver. For the 3×3 cube, God's Number is **20** (proved in 2010 by Rokicki, Kociemba, Davidson and Dethridge).

So "≤20-move solutions" is the paper's way of asking *how often does the solver hit the theoretically optimal range?* Baseline Kociemba does it 28% of the time; with multi-solution search that jumps to 57.5%. Note that the paper does *not* prove its solver is always optimal — it just lands in the optimal range much more often.

17 What is the anytime mode?

Anytime mode is a softer version of multi-solution search. Instead of a hard timeout, the caller gives a *budget* (for example, 0.5 seconds). The solver keeps finding better and better solutions within that budget and returns the best one at the deadline. If no solution has been found at all, you get a timeout error.

This mode matters for interactive apps — you cannot tell a user "please wait 25 seconds". The paper's anytime table (`results/anytime_sweep.json`) shows that at a 0.5-second budget the solver completes about 66% of cubes with a mean of 20.42 moves.

18 What is the neural part trying to do?

The neural part is *not* a solver. It is a small AI model (about 84 K parameters) whose only job is to rank the 18 possible moves at a node, so the solver can try the highest-ranked move first. The rest of the solver is unchanged: pruning tables, IDA*, Phase 1/Phase 2, everything.

The paper is very careful about why it does not use the neural network as a *heuristic* (i.e. as a replacement lower bound). Doing that would break Kociemba's admissibility guarantee and make the search dramatically slower on some cubes. Using the neural output only for move ordering keeps the solver correct.

19 What does "accuracy" mean here?

Accuracy means: "on a held-out test set, how often does the model's top-ranked move match the label we trained it to predict?" The paper reports three regimes:

Setup	Top-1	Top-3
9 scalar features (50 K cubes)	5.9%	—
161 binned features (100 K cubes, proxy)	10.8–13.4%	24.9–28.1%
161 binned features (2 M cubes, pruning-greedy)	**43.94%**	**64.48%**

Random would be 1/18 ≈ 5.56%. So scalar features are at chance. Rich binned features with pruning-greedy labels jump to nearly 44%.

The paper's key surprise: that big jump in accuracy does *not* translate into a big end-to-end search improvement.

20 What are top-1 and top-3 accuracy?

- **Top-1**: the model's first guess is correct.
- **Top-3**: the correct answer is in the model's best three guesses.

A model can be low in top-1 but decent in top-3, which still helps if the search tries the top few moves first before falling back to the rest.

21 What is a LUT and why did the authors build one?

LUT means **Look-Up Table**. Running the neural network every time the solver visits a node is expensive ($\sim$16 microseconds per query on the paper's laptop). That cost, multiplied by millions of nodes per solve, eats any gain from better move ordering.

The LUT is a clever shortcut: *offline*, the authors run the neural network once for every one of the 32 768 possible coordinate bins and store the resulting move rankings in a single 576 KB file. *Online*, each node just does a 1.5-microsecond array lookup — no neural net, no matrix multiplication. This is a simple form of **knowledge distillation**.

> *Neural forward pass: $\sim 16\,\mu s$.* *LUT lookup: $\sim 1.5\,\mu s$.*

22 Did the LUT fix the problem?

Partly. It killed the per-node inference cost completely. But the end-to-end node reduction over plain $K = 5$ was only $\approx 1.13\times$ (6.79 M $\rightarrow$ 6.02 M nodes), and the LUT variant actually solved one fewer cube within the 5-second timeout (199/200 vs. 200/200).

So the lesson is:

> *Inference cost was part of the problem, but the bigger problem is that Kociemba's pruning tables already order moves nearly as well as any admissibility-preserving heuristic can. There is very little room left to improve.*

23 What is the "negative result"?

A **negative result** is an experiment whose outcome was not the improvement you hoped for — but reporting it honestly is still valuable, because it tells the community what *not* to try.

Here the negative result is:

- A 7$\times$ jump in prediction accuracy (5.9% $\rightarrow$ 43.9%) delivered only a 1.13$\times$ node reduction.
- A bigger model (MLP-4 Wide, 257 K parameters instead of 84 K) gained only +2.6 percentage points of accuracy at +37% inference cost.
- A carefully retrained Δh predictor (v1 and v2) ends up exactly matching the class prior — the model learned *nothing useful beyond the marginal distribution.*

All three are honest signals that lightweight neural move ordering does not pay off against Kociemba's pruning tables in this regime.

24 What is the "headroom" measurement?

To prove that this is not just "the cube is already optimally solved", the authors directly measured how much better an *oracle* move ordering could be. On 6 000 randomly walked Phase 1 states (`results/headroom_measurement.json`):

- Kociemba's default lexicographic first-move is *progressing toward the goal* only $p_{\text{lex}} = 19.6\%$ of the time.
- The minimum-h child (the oracle) is progressing $p_{\text{top}} = 65.2\%$ of the time.
- That is a gap of 0.456 — a theoretical ceiling of roughly 10× node reduction if a solver could actually use the oracle.

So the room to improve exists. It is just not reachable by a small MLP on 161 binned features.

25 Why does the small MLP fail to close that gap?

Two independent retraining experiments are reported:

- **v1:** 150 K walk-sampled states, 161-dim features, three-way Δh cross-entropy. The model's progressing rate is 20.2%, indistinguishable from the 19.8% baseline.
- **v2:** 3 M states (20× more), per-coordinate embedding tables (∼11× more parameters), same training objective. Rate is 20.2% again. No movement.

Both converge *to the marginal class prior*, meaning the model learns how common each outcome is overall but nothing about move-conditional structure. The paper calls this a **feature-resolution ceiling**: 161 binned features are rich enough for state-level quantities like h, but too lossy to recover the permutation action of the move tables.

26 What is the `h_sort` experiment?

To separate "the oracle is unreachable" from "the headroom is fake", the authors also implemented an *oracle* move ordering: for every legal child, call the pruning tables and sort by h ascending. Results (`results/h_sort_benchmark.json`, `hsort_c_backend.json`):

- Python $K = 5$: 1.66× node reduction over default — real, but does not become a wall-clock win because the 36 extra pruning-table lookups per node cost more than the nodes saved.
- C backend $K = 1$: 11% node reduction but 13% slower wall-clock.

So the headroom is real, but not monetisable into faster solves — and certainly not by a small neural model.

27 What is a "hard case", and how does the paper handle it?

Hard cases are deliberately difficult scrambles — the paper uses 49 depth-20 scrambles and the superflip. The benchmark (`results/hard_cases.json`) shows that on these hard cubes the $K = 5$ improvement is even bigger: ≤20-move rate goes from 45% (baseline) to 71% ($K = 5$), at 71× more wall time. Multi-solution search scales the right way — it helps more where it is needed most.

28 Multi-seed validation — is the improvement real?

A classic concern with a single benchmark is: "maybe seed 42 is lucky." The paper runs the same comparison on four different seeds (42, 123, 456, 789) with 200 cubes each. Every single seed shows $K = 5$ beating baseline. Pooled across all 800 cube pairs:

- Mean improvement: $\Delta = 0.40 \pm 0.09$ moves.
- Cohen's $d = 0.59$ (a medium-to-large effect).
- 95% confidence interval: $[0.35, 0.45]$ moves.
- $p < 0.001$.

Robust, not a fluke.

29 What about the statistical terms?

Mean vs. median. Mean is the arithmetic average; median is the middle value. On timing data, a few very slow runs can pull the mean up without affecting the median. The paper reports both for timings and just the mean for move counts.

p-value. A small p-value means "the difference we saw is very unlikely to be pure chance". For $K = 1$ vs. $K = 5$, $p < 10^{-20}$ — well beyond any reasonable bar.

Effect size (d_z, d). Separate question: *how big* is the difference, measured in standard-deviation units? $d_z = 0.79$ is a large within-subject effect. $d = 0.59$ pooled across seeds is a medium-to-large between-subject effect.

Confidence interval. A plausible range for the true mean improvement. $[0.44, 0.62]$ moves on seed 42 tells you the real improvement is almost certainly between half a move and a bit over half a move — always positive.

30 What is the difference between the C backend and the Python backend?

The solver ships in two implementations:

- A fast C implementation (used when $K = 1$ with no neural or anytime): $\sim$10 ms per solve.
- A pure-Python implementation that supports all the enhanced features ($K > 1$, anytime, neural, h_sort): $\sim$59$\times$ slower.

For research results, everything is measured on the Python backend so the node counts are reproducible. For end-user wall-clock, the C backend is used in baseline mode and makes $K = 1$ essentially instantaneous in the desktop app.

31 How does this compare to DeepCubeA?

DeepCubeA is a large deep-reinforcement-learning cube solver (∼500 K parameters, ∼2 GPU-days to train, ∼10 seconds per solve, near-optimal solutions in 60% of cases). It uses weighted A* with a learned value network.

This paper is positioned as a complement, not a rival:

- Tiny model (84 K parameters, not 500 K).
- Minutes of CPU training, not days of GPU.
- Runs on a laptop, not a workstation.
- Used as an *add-on* to a classical solver, not a replacement.

The headline takeaway — that lightweight neural guidance does not add much on top of strong pruning tables — is a genuinely new observation, because previous deep-cube work (including DeepCubeA) replaces the classical solver rather than sitting on top of it.

32 What does the paper ultimately conclude?

Four bullet points, all directly supported by the frozen artifacts:

1. Multi-solution Phase 1 search is a simple, zero-training improvement that produces shorter solutions more often.
2. Representation and label quality drive move-prediction accuracy; architecture does not.
3. LUT distillation makes per-node neural inference effectively free — an engineering win that cleanly separates "prediction cost" from "search-space cost".
4. Even so, learned move ordering plateaus when the classical heuristic is already strong. Predict accuracy is not a sufficient measure of usefulness inside search.

The authors' recommended default for practitioners is $K = 3$: the best balance of solution quality and compute cost.

33 The simplest possible summary you can say in class

> "This paper improves a Rubik's Cube solver by trying several possible halfway states instead of only the first one. That simple change gives shorter solutions much more often — from 28% optimal to nearly 58%. The paper also tests a small neural network to guide the search. Even though prediction accuracy improves a lot, the final solver improves only a little, because Kociemba's pruning tables are already very strong. So the main lesson is that a simple search improvement works better than lightweight AI guidance in this setting — and that is a useful, honestly reported negative result in its own right."

34 Five lines to memorise

1. The paper extends Kociemba's Two-Phase Rubik's Cube solver.
2. The main algorithmic idea is to try $K > 1$ Phase 1 endpoints and keep the shortest combined solution.

3. At $K = 5$ the mean solution length drops from 20.77 to 20.23 moves ($p < 10^{-20}$, Cohen's $d_z = 0.79$).
4. The fraction of $\leq$20-move solutions roughly doubles, from 28.0% to 57.5%.
5. A neural move predictor lifts raw accuracy from 5.9% to 43.9%, but the end-to-end node reduction is only 1.13× — the pruning tables leave almost no room to improve.

35 Likely viva questions with beginner-friendly answers

What is the main contribution? Trying multiple Phase 1 endpoints and keeping the best combined solution.

Why does that help? Different Phase 1 endpoints can lead to Phase 2 sequences of different lengths, so taking the minimum of several tries gives shorter total solutions on average.

What is the AI part doing? Only ranking the 18 candidate moves at each search node, so the solver tries more promising moves first. It is not a solver by itself.

Did the AI beat the classical solver? No. In the final end-to-end benchmark the neural variant matches plain $K = 5$ on solution quality and is marginally less reliable (solves one fewer cube within the 5-second budget).

Then why include the AI part? Because it answers an important research question: *when* does learned move ordering help a classical solver, and *when* does it not? The paper gives a precise, measurable answer.

What is the practical recommendation? Use multi-solution search, preferably $K = 3$ (best $\leq$20-move rate) or $K = 5$ (shortest mean solution). Skip neural move ordering unless you have a much bigger model budget and finer features.

How big is the $K = 5$ effect and is it statistically strong? -0.53 moves on average, $p < 10^{-20}$, Cohen's $d_z = 0.79$, 95% confidence interval $[0.44, 0.62]$ moves. Replicated across four seeds with a pooled $\Delta = 0.40 \pm 0.09$. Very strong.

What is the anytime mode? A $K = 5$ solver with a user-specified time budget that returns the best solution found within the budget. Designed for interactive apps.

What is a LUT and why is it useful? A precomputed table of neural move rankings. It drops per-node inference cost from $\sim$16 μs to $\sim$1.5 μs, so inference cost stops being the bottleneck. The remaining bottleneck is that there is not much room to improve over Kociemba's pruning-table ordering.

What is God's Number? Twenty. The fewest moves needed to solve any cube using the optimal solver. This is why the paper tracks $\leq$20-move solution rate.

What is the headroom, and why does it matter? On 6 000 walk-sampled states the oracle move ordering progresses 65.2% of the time while Kociemba's default progresses only 19.6% of the time. That 0.456 gap implies a theoretical $\sim$10× node-reduction ceiling. The neural MLP captures almost none of it: that is the paper's feature-resolution ceiling claim.

Is this work reproducible? Yes. Every number in the paper traces to a named JSON in `results/`, all the code is public, models are shipped, and scrambles are deterministic under the advertised random seeds.

36 What to focus on first

Do not try to absorb the math on first reading. The learning order is:

1. What the paper is trying to improve (Kociemba's solver).
2. What Phase 1 and Phase 2 mean.
3. What K means.
4. The headline numbers (Section 14): $20.77 \rightarrow 20.23$ and $28\% \rightarrow 57.5\%$.
5. Why the neural part does not help much (pruning tables are strong; feature resolution is lossy).

Once those five points feel natural, come back and read the statistical sections and the `h_sort`/headroom discussion. Keep the memorised five lines handy during the viva — most follow-up questions reduce to one of them.